# Studio Nx

백엔드 155 — reset_password()

2026-10-14

# 백엔드 155 — `reset_password()` (새 비밀번호로 바꾸기)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/services/password_reset_service.py`, 오류 코드 `core/errors.py`의 `INVALID_RESET_LINK` |
| 작업 | P1-28 비밀번호 찾기 (FR-118): 여섯 번째 함수 |
| 작성 | Claude (AI), 2026-10-14, 사용자 검토 |
| 이전 | 154 `request_password_reset()` |
| 시험 | `tests/test_password_reset_service.py`의 10개 (이 파일 전체 39개) |
| 다음 | 이전 로그인 무효화: 토큰의 발급 시각(`iat`)이 `password_changed_at`보다 앞서면 거부 |

## 1. 하는 일

메일 링크의 토큰과 새 비밀번호를 받아 비밀번호를 바꿉니다.

| 순서 | 일 | 실패하면 |
|---|---|---|
| 1 | 토큰의 해시로 행을 찾고, 만료, 사용 여부, 회원 상태(`ACTIVE`)를 확인합니다 | 400 `INVALID_RESET_LINK` ("This link is invalid or has expired.") |
| 2 | 새 비밀번호 규칙(FR-102)을 검사합니다 | 422 + `details.new_password`. **토큰은 쓰지 않은 채로 둡니다** |
| 3 | 토큰을 한 번만 쓰도록 사용 처리합니다(`UPDATE … WHERE used_at IS NULL`, 바뀐 행이 1이어야 함) | 400 `INVALID_RESET_LINK` |
| 4 | 새 해시 저장, `password_changed_at` 기록, 이 회원의 남은 토큰도 모두 사용 처리, `commit` | |

## 2. 코드 (요약)

```
def reset_password(db: Session, token: str, new_password: str) -> None:
    try:
        token_hash = hash_reset_token(token)
    except ValueError:
        raise _invalid_link() from None

    now = datetime.now(UTC).replace(tzinfo=None)
    row = db.scalar(select(PasswordResetToken).where(PasswordResetToken.token_hash == token_hash))
    if row is None or row.used_at is not None or row.expires_at <= now:
        raise _invalid_link()
    user = db.get(User, row.user_id)
    if user is None or user.status != UserStatus.ACTIVE:
        raise _invalid_link()

    failed = check_password_rules(new_password)
    if failed:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Check the password requirements.",
                       details={"new_password": failed})
```

```
    claimed = db.execute(update(PasswordResetToken)
        .where(PasswordResetToken.token_id == row.token_id, PasswordResetToken.used_at.is_(None))
        .values(used_at=now))
    if claimed.rowcount != 1:
        db.rollback()
        raise _invalid_link()

    user.password_hash = hash_password(new_password)
    user.password_changed_at = now
    db.execute(update(PasswordResetToken)
        .where(PasswordResetToken.user_id == user.user_id, PasswordResetToken.used_at.is_(None))
        .values(used_at=now))
    db.commit()
```

## 3. 왜 이렇게

| 선택 | 이유 |
|---|---|
| **틀림, 만료, 이미 씀, 탈퇴 계정이 모두 같은 오류** | 이유를 알려 주면 "이 토큰은 한때 유효했다" 같은 정보를 추측하게 됩니다. 사용자에게는 "링크가 잘못되었거나 만료되었습니다. 다시 요청하세요"로 충분합니다 |
| **링크 검사가 비밀번호 규칙 검사보다 먼저** | 링크가 틀렸으면 비밀번호가 약해도 링크 오류가 먼저 나와야, 링크 문제가 가려지지 않습니다 |
| **비밀번호 규칙이 틀려도 토큰은 쓰지 않음** | 사용자가 화면에서 비밀번호만 고쳐서 다시 보낼 수 있습니다. 토큰을 먼저 써 버리면 약한 비밀번호 한 번에 링크가 죽어서 다시 메일을 받아야 합니다 |
| **`UPDATE … WHERE used_at IS NULL`로 한 번만 쓰기** | 조회(`SELECT`)와 사용 처리 사이에 같은 링크를 동시에 누른 두 요청이 모두 통과하는 틈이 있습니다. 조건부 `UPDATE`는 DB가 한 번에 처리해서 하나만 1행을 바꾸고, 나머지는 0행이라 막힙니다 |
| 이 회원의 남은 토큰도 모두 사용 처리 | 비밀번호를 바꿨는데 이전에 받은 다른 링크가 살아 있으면 그것으로 또 바꿀 수 있습니다 |
| `password_changed_at` 기록 | 이 시각보다 먼저 발급된 로그인(액세스, 리프레시 토큰)을 무효로 만드는 데 씁니다. 그 검사는 다음 함수입니다 |
| 현재 비밀번호와 같은 값을 막지 않음 | 비밀번호 변경(FR-107)은 같은 값을 막지만, 재설정은 잊어서 오는 흐름이라 의미가 없습니다 |
| 새 오류 코드 `INVALID_RESET_LINK` | 화면(JS)이 이 코드를 보고 "다시 요청" 안내로 분기합니다 |

## 4. 시험 (10개 추가, 파일 전체 39개 통과)

| 확인 | 결과 |
|---|---|
| 유효한 토큰으로 비밀번호가 바뀌고, 예전 비밀번호는 못 쓰고, `password_changed_at`이 기록됨 | ✓ |
| 토큰은 한 번만 쓸 수 있음(두 번째는 400, 반영 안 됨) | ✓ |
| 없는 토큰, 만료, 이미 씀, 빈 값이 **같은 오류, 같은 문구** | ✓ |
| 약한 비밀번호는 422이고 토큰, 비밀번호가 그대로이며 고쳐서 다시 보내면 됨 | ✓ |
| 링크가 틀리면 비밀번호가 약해도 링크 오류가 먼저 | ✓ |
| 탈퇴, 정지 회원은 재설정 불가 (2개) | ✓ ✓ |
| 이 회원의 다른 살아 있는 토큰이 모두 무효가 됨 | ✓ |
| 다른 회원은 영향을 받지 않음 | ✓ |
| **조회 뒤 다른 요청이 먼저 토큰을 쓴 경우**(경쟁): 같은 오류, 비밀번호 그대로 | ✓ |

**시험을 확인하는 시험.** 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

| 바꾼 것 | 결과 |
|---|---|
| 만료 검사 제거 | 1개 실패 |
| `password_changed_at` 기록 제거 | 1개 실패 |
| 남은 토큰 무효화 제거 | 1개 실패 |
| 상태(`ACTIVE`) 검사 제거 | 2개 실패 |
| 사용 처리 행 수(`rowcount`) 확인 제거 | 1개 실패 |
| **"이미 씀" 검사 제거** | **실패 없음** |

마지막은 시험의 빈틈이 아니라 **중복 방어**입니다. "이미 씀" 검사를 지워도 `UPDATE … WHERE used_at IS NULL`의 행 수 확인이 같은 요청을 막아서 동작이 같기 때문입니다. 두 검사를 함께 둔 것은 첫 검사가 빠른 거절(쓸데없이 비밀번호 규칙 검사와 해시를 하지 않음)을, 두 번째가 경쟁 상태의 안전망을 맡도록 한 것입니다.

전체 pytest 740개 통과, `ruff` 통과.

## 5. 다음 함수에 남기는 주의

`password_changed_at`은 밀리초까지 저장하지만 JWT의 발급 시각(`iat`)은 **초 단위**입니다. 비교할 때 초 미만을 버리지 않으면, 재설정 직후 같은 초 안에 새로 로그인한 토큰이 "이전 로그인"으로 잘못 거부될 수 있습니다. 다음 함수에서 이 경계를 시험으로 확인합니다.